

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Ulepszony procesor

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 12

Studia dzienne

Informatyka

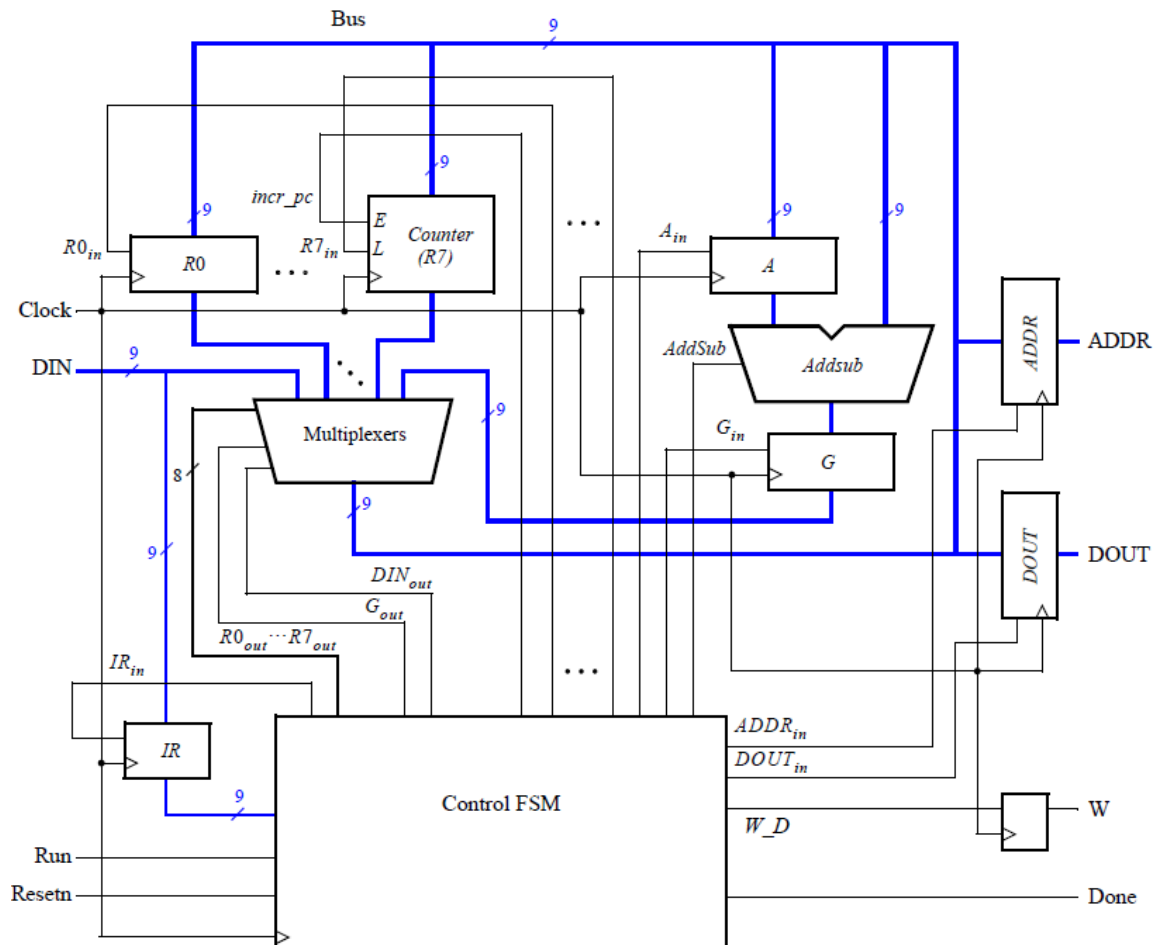
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 02.06.2022

Zadanie 1

Utwórz projekt processor_2 ulepszonej wersji procesora, którego strukturę pokazano na rys. 1.



Rys. 1. Ulepszona wersja procesora.

Zestaw komend nowego procesora jest rozszerzony o następujące komendy:

Tabela. 1. Nowe instrukcje wykonywane w procesorze

Operation	Function performed
$ld\ Rx, [Ry]$	$Rx \leftarrow [[Ry]]$
$st\ Rx, [Ry]$	$[Ry] \leftarrow [Rx]$
$mvnz\ Rx, Ry$	if $G \neq 0$, $Rx \leftarrow [Ry]$

gdzie **ld (load)** – zapisuje dane do rejestru **Rx** z komórki pamięci, której adres jest wskazany w rejestrze **Ry**; **st (store)** – zawartość rejestru **Rx** zapisuje w komórce pamięci, której adres jest określony w

rejestrze **Ry**; **mvnz (move if not zero)** zapisuje do rejestru **Rx** zawartość rejestru **Ry**, jeżeli zawartość rejestru **G** nie wynosi zero (to znaczy, że jest wykonywane **mv**, jeśli **G ≠ 0**).

W strukturze na rys. 1 rejestr R7 jest zastąpiony przez licznik programu (PC), który wskazuje adres polecenia w pamięci. Po zresetowaniu procesora licznik ustawiony jest na adres 0. Na początku każdego polecenia PC jest używany jako adres polecenia w pamięci. Następnie polecenie jest przechowywane w rejestrze instrukcji (rejestr IR), a PC jest automatycznie zwiększany, aby wskazać następne polecenie. Podczas wykonywania polecenia **mvi** rejestr PC wskazuje adres danych, a następnie ponownie zwiększa się kolejno.

Sygnał sterujący **incr_PC** jest sygnałem zezwolenia licznika PC. Do PC można również załadować dowolną wartość za pomocą polecenia **mv** lub **mvi**. W tym przypadku sygnał sterujący **R7in** jest wykorzystywany do równoległego załadowania licznika. Bieżąca zawartość PC może zostać skopiowana do innego rejestru za pomocą polecenia **mv**. Na przykład poniższy fragment kodu implementuje opóźnienie za pomocą pętli, która może być używana w dużym programie do tworzenia opóźnień:

```
mvi    R2,#1
mvi    R4,#10000000    % wartość opóźnienia binarnie
mv     R5,R7            % zapisz adres następnej instrukcji
sub    R4,R2            % dekrementacja wartości opóźnienia
mvnz   R7,R5            % kontynuacja odejmowania,
                        % dopóki wartość opóźnienia nie osiągnie 0
```

Odczytywanie danych z pamięci zewnętrznej i zapisywanie w pamięci odbywa się za pomocą rejestrów **ADDR** i **DOUT**, a także sygnałów kontrolnych **ADDRin**, **DOUTin** i **W_D**. Aby odczytać dane z pamięci wystarczy umieścić adres czytelnej komórki w rejestrze **ADDR**. Aby odczytać (wybrać) polecenie z pamięci, wartość licznika poleceń PC umieszczona jest w rejestrze **ADDR**. Dane lub polecenia odczytane z pamięci są wysyłane do procesora za pośrednictwem szyny **DIN**. Aby zapisać do pamięci, adres komórki znajduje się w rejestrze **ADDR**, a dane, które mają być zapisane w rejestrze **DOUT**, ustawiony jest sygnał **W_D**.

Wskazówka: stwórz moduł **counter_9_bits_on_sload** 9-bitowego licznika z wejściem aktywującym i możliwością synchronicznego ładowania danych. Użyj instancji tego modułu zamiast rejestru R7. Wykonaj symulację projektu **procesor_2**.

Kod w Verilog

Moduł procesora

```
module processor_2(
    input [8:0] DIN,
    input Resetn, Clock, Run,
    output reg Done,
    output reg [8:0] BusWires,
    output [8:0] ADDR, DOUT,
    output W);

    // stany FSM
    localparam [2:0] T0 = 3'b000, T1 = 3'b001, T2 = 3'b010, T3 = 3'b011,
        T4 = 3'b100, T5 = 3'b101;
    // kody instrukcji
    localparam [2:0] mv = 3'b000, mvi = 3'b001, add = 3'b010, sub = 3'b011,
```

```

ld = 3'b100, st = 3'b101, mvnz = 3'b110;
wire [0:7] Xreg, Yreg;
wire [2:0] I;
wire [9:1] IR;
wire [8:0] A, G;
wire [8:0] R0, R1, R2, R3, R4, R5, R6, PC;
reg G_in, G_out, DIN_out, AddSub, IR_in, A_in;
wire [8:0] Sum;
reg [2:0] Tstep_Q /* synthesis keep */;
reg [2:0] Tstep_D;
reg [0:7] R_in, R_out;
reg incr_PC;
wire zero_G = (G == 9'b000000000) /* synthesis keep */;
reg ADDR_in, DOUT_in;
reg W_D;

assign I = IR[9:7];
dec3to8 decX(IR[6:4], 1'b1, Xreg);
dec3to8 decY(IR[3:1], 1'b1, Yreg);

// sterowanie przejściami między stanami FSM
always @ (Tstep_Q, Run, Done)
    case (Tstep_Q)
        T0:
            if (~Run) Tstep_D = T0;
            else Tstep_D = T1;
        T1:
            if (Done) Tstep_D = T0;
            else Tstep_D = T2;
        T2:
            if (Done) Tstep_D = T0;
            else Tstep_D = T3;
        T3:
            if (Done) Tstep_D = T0;
            else Tstep_D = T4;
        T4:
            if (Done) Tstep_D = T0;
            else Tstep_D = T5;
        default: // T5
            Tstep_D = T0;
    endcase

// sterowanie wyjściami FSM
always @ (Tstep_Q, I, Xreg, Yreg, zero_G) begin
    Done = 1'b0;
    G_in = 1'b0;
    G_out = 1'b0;
    A_in = 1'b0;
    AddSub = 1'b0;
    DIN_out = 1'b0;
    R_in = 8'b0000_0000;
    R_out = 8'b0000_0000;
    IR_in = 1'b0;
    incr_PC = 1'b0;
    ADDR_in = 1'b0;
    DOUT_in = 1'b0;
    W_D = 1'b0;
    case (Tstep_Q)
        T0: begin // wyjścia z FSM w takcie 0
            R_out[7] = 1'b1;
            ADDR_in = 1'b1;
        end
    end
end

```

```

// na zboczach T0 -> T1 zegara zlecamy RAMowi wystawienie kodu instrukcji
T1: begin // wyjścia z FSM w takcie 1
    // RAM dopiero na zboczach T1 -> T2 będzie miał gotowe odczytane słowo
    incr_PC = 1'b1;
end
// na zboczach T1 -> T2 RAM inicjuje podanie odczytanego słowa na DIN
T2: begin // wyjścia z FSM w takcie 2
    // w wyniku opóźnień chwilę po zboczach T1 -> T2 (ale przed T2 -> T3) na DIN trafia
słowo
    IR_in = 1'b1;
end
// na zboczach T2 -> T3 procesor inicjuje zapisanie w IR słowa z DIN
T3: // wyjścia z FSM w takcie 3
    // w wyniku opóźnień chwilę po zboczach T2 -> T3 (ale przed T3 -> T4) do IR trafia kod
instrukcji
    case (I)
        mv: begin
            R_out = Yreg;
            R_in = Xreg;
            Done = 1'b1;
        end
        mvi: begin
            R_out[7] = 1'b1;
            ADDR_in = 1'b1;
            incr_PC = 1'b1;
        end
        add, sub: begin
            R_out = Xreg;
            A_in = 1'b1;
        end
        ld, st: begin
            R_out = Yreg;
            ADDR_in = 1'b1;
        end
        mvnz: begin
            Done = 1'b1;
            if (~zero_G) begin
                R_out = Yreg;
                R_in = Xreg;
            end
        end
    endcase
// w instrukcjach mvi i ld na zboczach T3 -> T4 zlecamy RAMowi wystawienie słowa danych
T4: // wyjścia z FSM w takcie 4
    case (I)
        // mvi, ld: RAM dopiero na zboczach T4 -> T5 będzie miał gotowe odczytane słowo
        add: begin
            R_out = Yreg;
            G_in = 1'b1;
        end
        sub: begin
            R_out = Yreg;
            G_in = 1'b1;
            AddSub = 1'b1;
        end
        st: begin
            R_out = Xreg;
            DOUT_in = 1'b1;
            W_D = 1'b1;
            Done = 1'b1;
        end
    endcase
endcase

```

```

        // w instrukcjach mvi i ld na zboczach T4 -> T5 RAM inicjuje podanie odczytanego słowa na
DIN
        T5: // wyjścia z FSM w takcie 5
            case (I)
                mvi, ld: begin
                    // w wyniku opóźnień chwilę po zboczach T4 -> T5 (ale przed T5 -> T0) na DIN
trafia słowo
                    DIN_out = 1'b1;
                    R_in = Xreg;
                    Done = 1'b1;
                end
                add, sub: begin
                    G_out = 1'b1;
                    R_in = Xreg;
                    Done = 1'b1;
                end
            endcase
        endcase
    end

// sterowanie przerzutnikami FSM
always @ (posedge Clock, negedge Resetn)
    if (~Resetn) Tstep_Q <= T0;
    else Tstep_Q <= Tstep_D;

// rejestry
regn #(9) reg_0(BusWires, R_in[0], Clock, R0);
regn #(9) reg_1(BusWires, R_in[1], Clock, R1);
regn #(9) reg_2(BusWires, R_in[2], Clock, R2);
regn #(9) reg_3(BusWires, R_in[3], Clock, R3);
regn #(9) reg_4(BusWires, R_in[4], Clock, R4);
regn #(9) reg_5(BusWires, R_in[5], Clock, R5);
regn #(9) reg_6(BusWires, R_in[6], Clock, R6);
counter #(9) cnt_pc(Resetn, incr_PC, R_in[7], Clock, BusWires, PC);
regn #(9) reg_A(BusWires, A_in, Clock, A);
regn #(9) reg_G(Sum, G_in, Clock, G);
regn #(9) reg_IR(DIN, IR_in, Clock, IR);
regn #(9) reg_ADDR(BusWires, ADDR_in, Clock, ADDR);
regn #(9) reg_DOUT(BusWires, DOUT_in, Clock, DOUT);
regn #(1) reg_W(W_D, 1'b1, Clock, W);

// alu
ripple_carry_adder_subtractor #(9) alu(A, BusWires, AddSub, Sum);

// multiplexer
wire [0:9] mux_s;
assign mux_s = {DIN_out, R_out, G_out};
always @ (*)
    case (mux_s)
        default: BusWires = DIN;
        10'b0100000000: BusWires = R0;
        10'b0010000000: BusWires = R1;
        10'b0001000000: BusWires = R2;
        10'b0000100000: BusWires = R3;
        10'b0000010000: BusWires = R4;
        10'b0000001000: BusWires = R5;
        10'b0000000100: BusWires = R6;
        10'b0000000010: BusWires = PC;
        10'b0000000001: BusWires = G;
    endcase
endmodule

```

Moduł N-bitowego rejestru

```
module regn
#(parameter N = 9) (
    input [N-1:0] R,
    input Rin, Clock,
    output reg [N-1:0] Q);

    initial Q = {N{1'b0}};
    always @ (posedge Clock)
        if (Rin) Q <= R;
        else Q <= Q;

endmodule
```

Moduł dekodera z kodu binarnego na 8-bitowy kod one-hot

```
module dec3to8(
    input [2:0] W,
    input En,
    output reg [0:7] Y);

    always @ (W, En)
        if (En == 1)
            case (W)
                3'b000: Y = 8'b10000000;
                3'b001: Y = 8'b01000000;
                3'b010: Y = 8'b00100000;
                3'b011: Y = 8'b00010000;
                3'b100: Y = 8'b00001000;
                3'b101: Y = 8'b00000100;
                3'b110: Y = 8'b00000010;
                3'b111: Y = 8'b00000001;
            endcase
        else
            Y = 8'b00000000;

endmodule
```

Moduł sumatora-subtraktora, który używamy jako jednostka arytmetyczno-logiczna (ALU)

```
module ripple_carry_adder_subtractor
#(parameter N = 4) (
    input [N-1:0] A, B,
    input sub,
    output [N-1:0] S);
    // output cout,
    // output overflow);

    wire [N-2:0] c;
    // assign overflow = c[N-2] ^ cout;
    generate
        genvar i;
        for (i = 0; i < N; i = i + 1)
            begin: ad
                case (i)
                    0: full_adder fa(A[i], sub ^ B[i], sub, S[i], c[i]);
                    N-1: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], /*cout*/);
                    default: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], c[i]);
                endcase
            end
        endgenerate
```

```
endmodule
```

Moduł pełnego sumatora

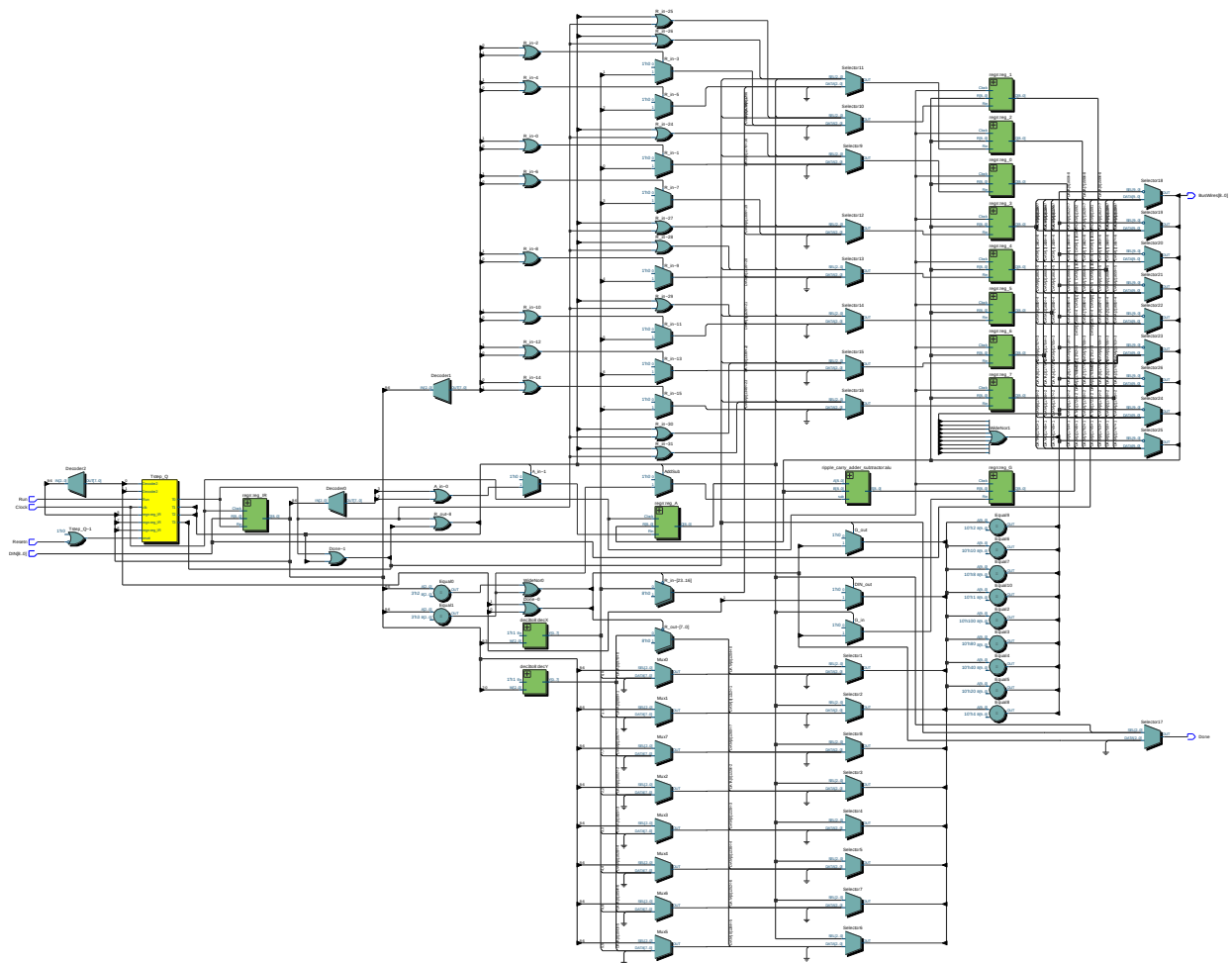
```
module full_adder(  
    input a, b, cin,  
    output s, cout);  
  
    assign s = cin ^ (a ^ b);  
    assign cout = a & b | (a ^ b) & cin;  
  
endmodule
```

Moduł N-bitowego licznika

```
module counter  
    #(parameter N = 4) (  
        input areset, enable, sload, clock,  
        input [N-1:0] data,  
        output reg [N-1:0] state);  
  
    initial state = {N{1'b0}};  
    always @ (posedge clock, negedge areset)  
        if (~areset) state = {N{1'b0}};  
        else if (sload) state <= data;  
        else if (enable) state <= state + 1'b1;  
        else state <= state;  
  
endmodule
```


Wyniki syntezy

RTL Viewer



State Machine Viewer układu sterowania

Graf stanów

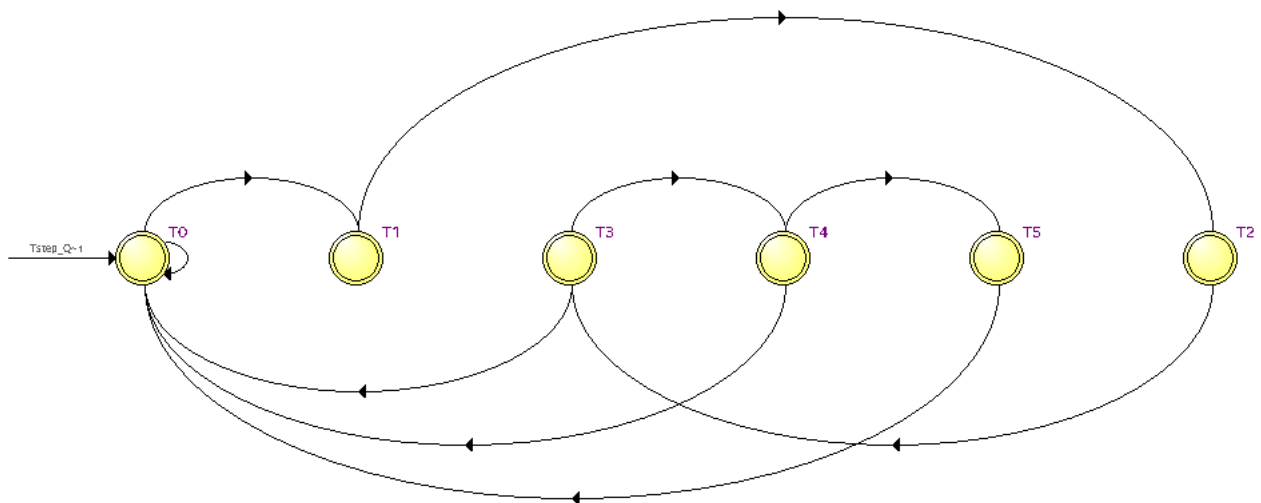


	Tabela przejść		
	Source State	Destination State	Condition
1	T0	T1	{Run}
2	T0	T0	{!Run}
3	T1	T2	
4	T2	T3	
5	T3	T4	{!Done}
6	T3	T0	{Done}
7	T4	T5	{!Decoder0}
8	T4	T0	{Decoder0}
9	T5	T0	

	Tabela kodowania						
	Name	T5	T4	T3	T2	T1	T0
1	T0	0	0	0	0	0	0
2	T1	0	0	0	0	1	1
3	T2	0	0	0	1	0	1
4	T3	0	0	1	0	0	1
5	T4	0	1	0	0	0	1
6	T5	1	0	0	0	0	1

Wyniki symulacji funkcjonalnej

Sygnały sterujące ustawione na 1 przy każdym takcie zegara dla każdej instrukcji są pokazane w poniższej tabeli.

		T0	T1	T2	T3	T4	T5
mv	I0	R7out, ADDRin	incr_PC	IRin	RYout, RXin, Done		
mvi	I1	R7out, ADDRin	incr_PC	IRin	R7out, ADDRin, incr_PC	(RAM jeszcze nie wystawił słowa na DIN)	DINout, RXin, Done
add	I2	R7out, ADDRin	incr_PC	IRin	RXout, Ain	RYout, Gin	Gout, RXin, Done
sub	I3	R7out, ADDRin	incr_PC	IRin	RXout, Ain	RYout, Gin, AddSub	Gout, RXin, Done
ld	I4	R7out, ADDRin	incr_PC	IRin	RYout, ADDRin	(RAM jeszcze nie wystawił słowa na DIN)	DINout, RXin, Done
st	I5	R7out, ADDRin	incr_PC	IRin	RYout, ADDRin	RXout, DOUTin, W_D, Done	
mvnz	I6	R7out, ADDRin	incr_PC	IRin	Done i jeżeli zero_G == 0, to: RYout, RXin		

W każdej instrukcji najpierw pobieramy jej kod z pamięci RAM w następujący sposób:

- Na zboczu T0 -> T1 zegara zapisujemy adres w rejestrze ADDR i zlecamy pamięci wystawienie kodu instrukcji spod podanego adresu. Dopiero na zboczu T1 -> T2 pamięć będzie miała przygotowane odczytane słowo do wystawienia na wejście DIN procesora.

- Na zboczu T1 -> T2 pamięć inicjuje podanie odczytanego słowa na DIN. W wyniku opóźnień słowo trafia na DIN dopiero chwilę po zboczu T1 -> T2 (ale przed T2 -> T3).

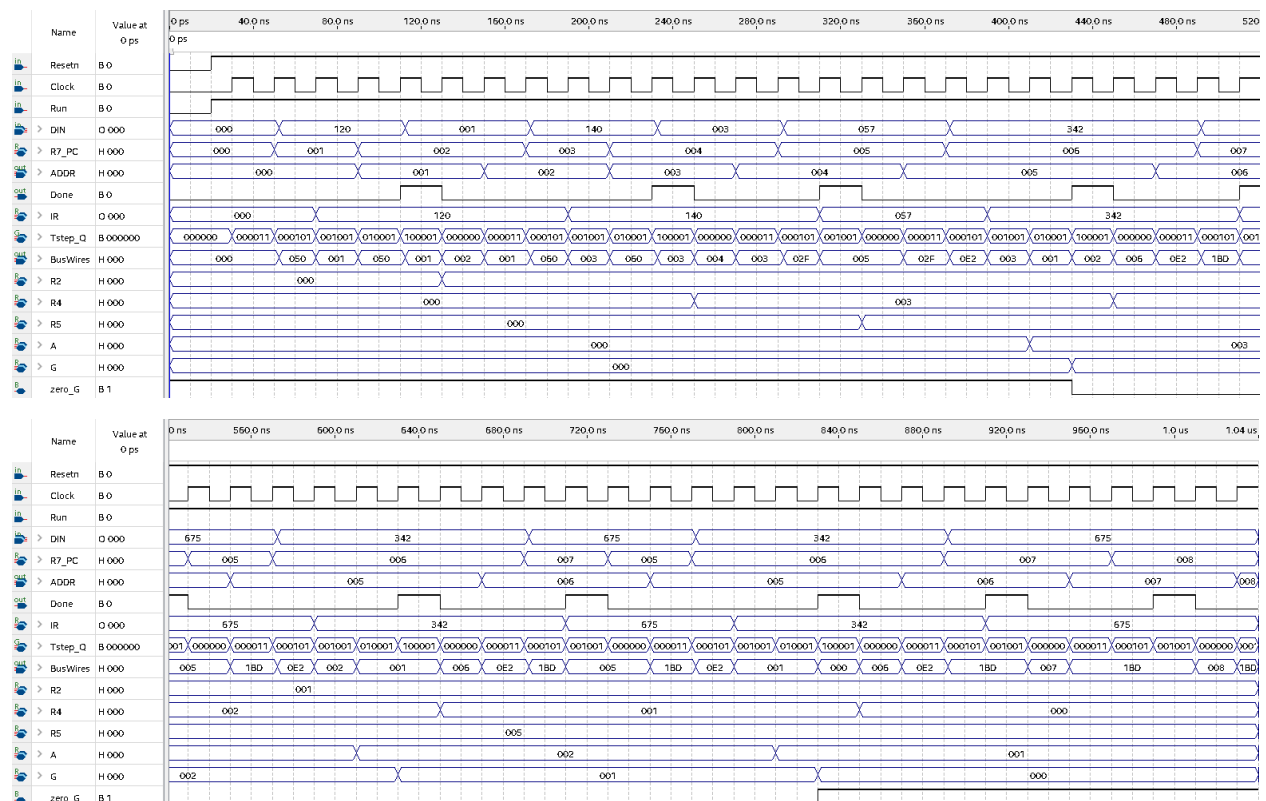
- Na zboczu T2 -> T3 procesor inicjuje zapisanie w rejestrze instrukcji (IR) słowa z DIN. W wyniku opóźnień słowo trafia do IR dopiero chwilę po zboczu T2 -> T3 (ale przed T3 -> T4).

Chwilę po rozpoczęciu taktu T3 kod instrukcji znajduje się w IR i układ sterowania zaczyna wykonywać instrukcję poprzez ustawianie sygnałów komponentów układu operacyjnego: multipleksera, rejestrów i ALU. W instrukcjach mvi i ld, oprócz pobrania kodu instrukcji, wykonujemy też odczyt danych z pamięci. Podobnie jak przy odczycie kodu instrukcji, słowo danych trafia na wejście DIN procesora dopiero chwilę po zboczu T4 -> T5.

Instrukcje wykonywane są w następujących przedziałach czasu:

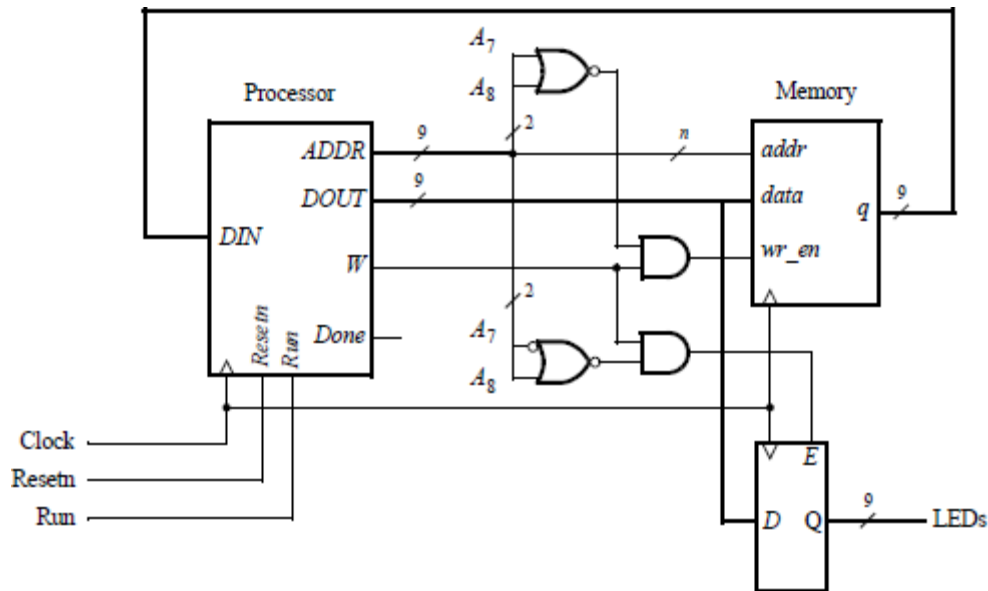
Instrukcja	Przedział czasu [ns]
mvi R2,#1	30 - 130
mvi R4,#3	150 - 250
mv R5,R7	270 - 330
sub R4,R2	350 - 450, 550 - 650, 750 - 850
mvnz R7,R5	470 - 530, 670 - 730, 870 - 930, 950 - 1010

Aby można było zobaczyć w symulacji działanie instrukcji mvnz przy G równym 0, inicjujemy R4 małą wartością 3 („mvi R4,#3”). Przy G równym 0 instrukcja mvnz nie nadpisuje licznika programu (R7_PC) adresem instrukcji „sub R4, R2” przechowywanym w rejestrze R5.



Zadanie 2

Utwórz project_2_with_RAM, którego strukturę pokazano na rys. 2.



Rys. 2. Poprawione połączenie procesora z pamięcią i rejestrem wyjściowym

Tutaj do procesora podłączone są dwa urządzenia zewnętrzne: pamięć Memory 128x9 i 9-bitowy rejestr. Pamięć służy do przechowywania poleceń i danych, a rejestr pozwala obserwować zawartość rejestru procesora DOUT na diodach LED. Magistrala adresowa (wyjście rejestru ADDR) ma 9 bitów. 7 najmniej znaczących bitów A[6: 0] służy do adresowania pamięci, a bity starsze A8 i A7 są używane do wybierania urządzenia zewnętrznego. Gdy A8A7 = 00, wybierana jest pamięć, a gdy A8A7 = 01, wybierany jest rejestr. Aby zaimplementować takie adresowanie w układzie na rys. 2, stosuje się układ wykorzystujący bramki logiczne.

Aby zaimplementować projekt, utwórz blok pamięci RAM128x9, korzystając z funkcji biblioteki z katalogu IP Catalog. Pobierz program za pomocą pliku MIF. Kod programu przedstawiono na rys. 3.

```
DEPTH = 128;
WIDTH = 9;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT
BEGIN
% Ten kod wyświetla licznik (w rejestrze R2) na czerwonych diodach LED. %
00 : 001001000; % mvi R1,#1 // inicjalizacja R %
01 : 000000001;
02 : 001010000; % mvi R2,#0 // licznik do wyświetlania na LEDs %
03 : 000000000;
04 : 001011000; % Loop mvi R3,#010000000 // R3 = adres rejestru LEDs %
05 : 010000000;
06 : 101010011; % st R2,R3 // zapis do LEDs %
```

```

07 : 010010001; % add R2,R1 // inkrementacja licznika dla LEDs %
08 : 001011000; % mvi R3,#11111111 // wartość opóźnienia %
09 : 111111111;
0A: 000101111; % mv R5,R7 // zapis adresu następnej instrukcji %
0B: 001100000; % Outer mvi R4,#11111111 // zagnieżdżony cykl opóźnienia %
0C: 111111111;
0D: 000000111; % mv R0,R7 // zapis adresu następnej instrukcji %
0E : 011100001; % Inner sub R4,R1 // dekrementacja zmiennej opóźnienia cyklu %
0F : 110111000; % mvnz R7,R0 // przedłużenie cyklu Inner jeśli R4 !=0 %
10 : 011011001; % sub R3,R1 // dekrementacja opóźnienia cyklu Outer %
11 : 110111101; % mvnz R7,R5 // przedłużenie cyklu Outer jeśli R3 !=0 %
12 : 001111000; % mvi R7,#Loop // ponowne wykonanie %
13 : 000000100;

END;

```

Rys. 3. Przykładowy program w pliku inicjującym pamięć (**MIF**)

Ten program pokazuje 8-bitową wartość licznika na diodach LED. Program zawiera pętle spowalniające zmiany licznika, aby móc obserwować jego wartości. Zobacz raporty TimeQuest Time Analyzer. Sprawdź, czy schemat procesu działa poprawnie przy częstotliwości 50 MHz. W przeciwnym razie użyj narzędzi Quartus, aby zmienić kod projektu na prędkość 50 MHz.

Użyj przerzutników, aby dopasować asynchroniczne wejście Run do zegara procesora. Użyj następujących przypisań pinów:

Wyprowadzenia	Piny
Run	SW9
Resetn	KEY0
clk	CLOCK_50
LEDs	LEDR8-0

Wykonaj symulację projektu i przetestuj go na płycie.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```

module processor_2_with_RAM_on_board(
    input [9:9] SW,
    input [0:0] KEY,
    input CLOCK_50,
    output [8:0] LEDR);

    processor_2_with_RAM proc(KEY[0], CLOCK_50, SW[9], LEDR);

endmodule

```

Moduł procesora z pamięcią RAM i rejestrem urządzenia zewnętrznego

```

module processor_2_with_RAM(
    input Resetn, Clock, ext_Run,
    output [8:0] LEDS);

    wire [8:0] q;
    wire [8:0] DIN = q, ADDR, DOUT /* synthesis keep */;

```

```

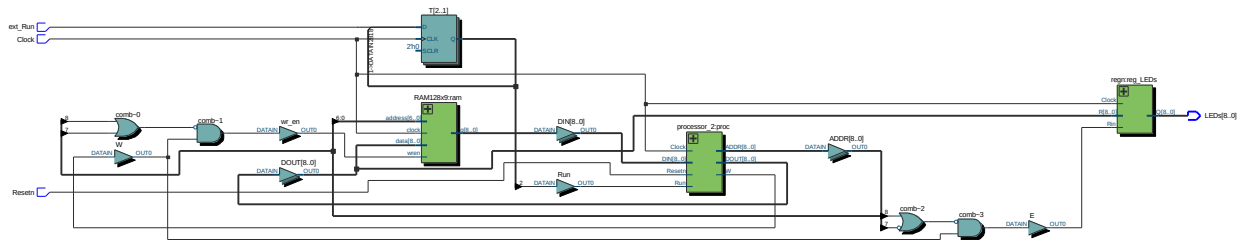
wire W /* synthesis keep */;
wire wr_en = (~(ADDR[7] | ADDR[8])) & W /* synthesis keep */;
wire E = (~((~ADDR[7]) | ADDR[8])) & W /* synthesis keep */;
reg #(9) reg_LEDs(DOUT, E, Clock, LEDs);
RAM128x9 ram(ADDR[6:0], Clock, DOUT, wr_en, q);
reg T1, T2 /* synthesis keep */;
always @ (posedge Clock) begin
    T2 <= T1;
    T1 <= ext_Run;
end
end
wire Run = T2 /* synthesis keep */;
processor_2 proc(DIN, Resetn, Clock, Run, ADDR, DOUT, W);

endmodule

```

Pozostałe moduły: processor_2, regn, dec3to8, ripple_carry_adder_subtractor, full_adder są takie same jak w zadaniu 1.

Wyniki syntezy



Przyjmujemy następującą zawartość pliku SDC dla Timing Analyzera:

```

create_clock -name CLOCK_50 -period "50 MHz" [get_ports {CLOCK_50}]
create_clock -name virt_CLOCK_50_in -period "50 MHz"
create_clock -name virt_CLOCK_50_out -period "50 MHz"
derive_pll_clocks -create_base_clocks
derive_clock_uncertainty
set_input_delay -clock { virt_CLOCK_50_in } -min 0 [get_ports {SW[*] KEY[*]}]
set_input_delay -clock { virt_CLOCK_50_in } -max 1 [get_ports {SW[*] KEY[*]}]
set_output_delay -clock { virt_CLOCK_50_out } -min 0 [get_ports {LEDR[*]}]
set_output_delay -clock { virt_CLOCK_50_out } -max 1 [get_ports {LEDR[*]}]

```

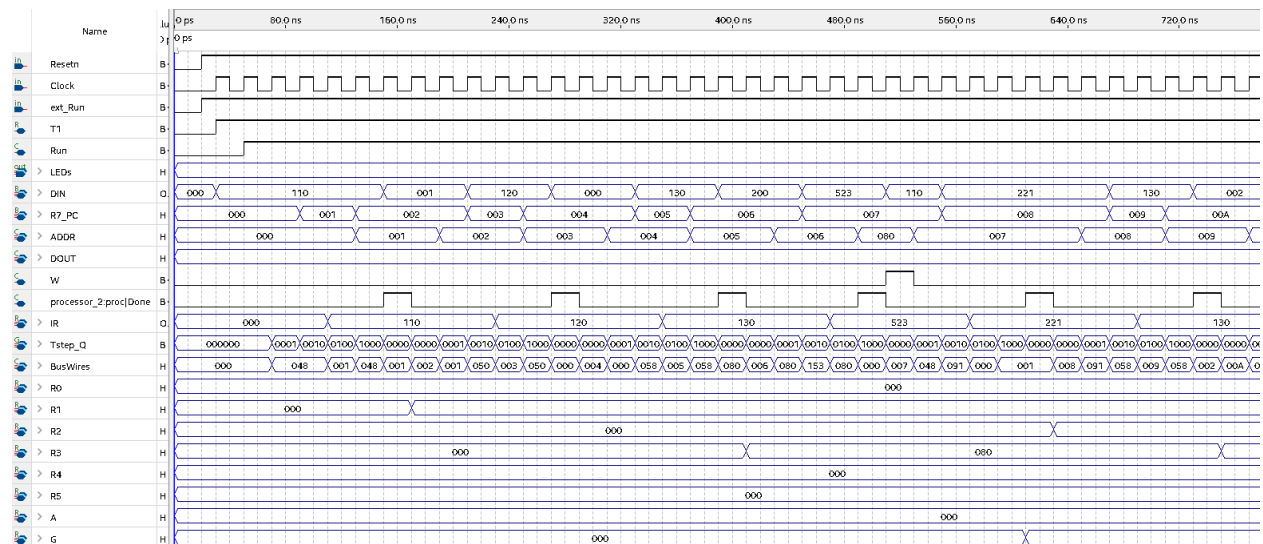
Według raportu Timing Analyzera układ działa prawidłowo z zegarem CLOCK_50 o częstotliwości 50 MHz.

Slow 1100mV 85C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	92.37 MHz	92.37 MHz	CLOCK_50	

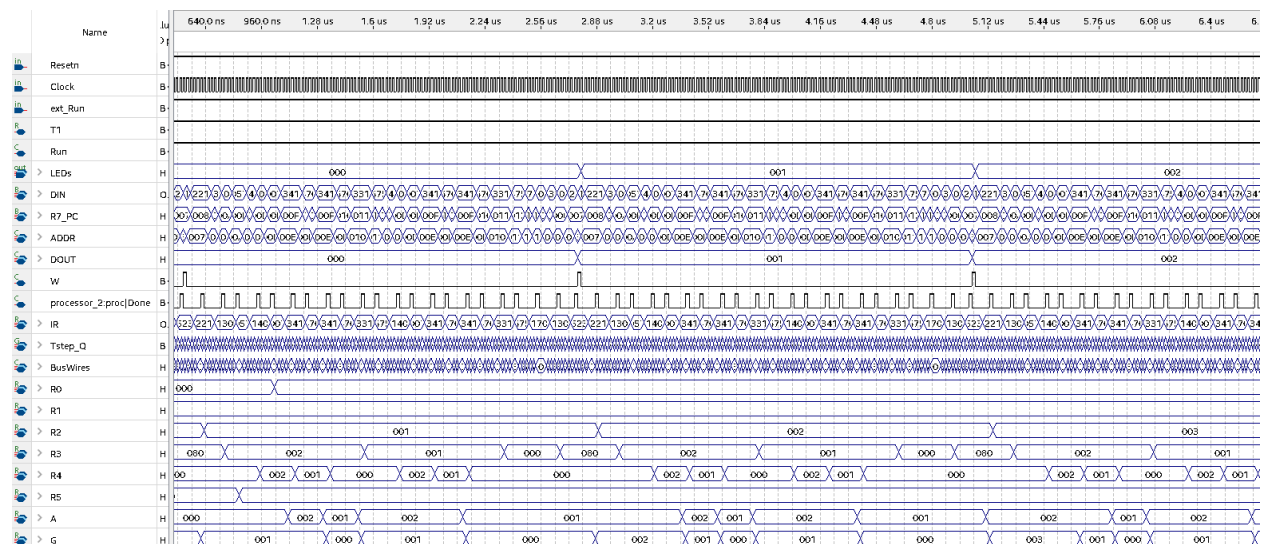
Wyniki symulacji funkcjonalnej

Inicjalizacja:

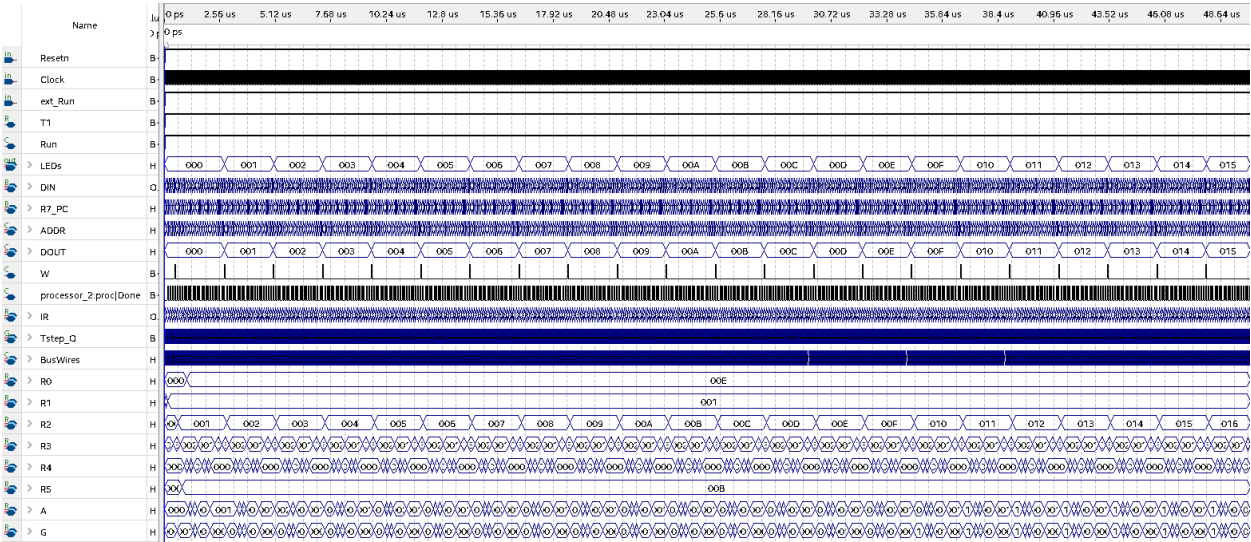
- jedynki do dekrementacji opóźnień i inkrementacji licznika R2 (mvi R1,#1),
- licznika wyświetlanego na LEDach (mvi R2,#0).



Początek odliczania. Przyjmujemy zewnętrzne i wewnętrzne opóźnienie o długości 2, aby można było zobaczyć działanie układu w symulacji.



Dalsze odliczanie.



Wnioski

Wszystkie zadania zostały zrealizowane.